Hazard Pointers with Reference Counter

Mathieu Desnoyers, EfficiOS



Kernel Recipes 2024
September 23-25
Paris, France

EfficiOS

Outline

- Existence Guarantees
- Mutual Exclusion (Locking)
- Read-Copy Update (RCU)
- Hazard Pointers (HP)
- Reference Counters (refcount)
- Existence Guarantees Tradeoffs
- Combining Existence Guarantees
- Hazard Pointers with Reference Counter (hpref)
- Benchmarks
- Future Work
- References

Existence Guarantees

- Immutable Data (Invariant, Mutual Exclusion)
- Read-Copy Update (RCU)
- Hazard Pointers (HP)
- Reference counters (refcount)

Mutual Exclusion (Locking)

- Spinlock
- Mutex
- Reader-Writer Lock
- Sequence Lock (readers retry if object is concurrently modified)

Read-Copy Update (RCU)

- RCU publication guarantee:
  - the object content stores are ordered before publishing a pointer to the object.
- RCU grace period guarantee:
  - Must observe all readers as going through a quiescent state (not in a read-side critical section) between unpublising pointer to object and object reclaim.
  - After unpublising pointer, wait for the end of pre-existing read-side critical sections before allowing reclaim.

Hazard Pointers (HP)

- Publication of HP similar to RCU,
- Track usage of specific pointers by readers through HP slots.

Hazard Pointers (HP): Reclaim

- High level view of unpublish/reclaim of object:
  - Store pointer to NULL (or other new object address)
  - Memory barrier
  - Check each HP slot for object address (load-acquire), wait until each of them is observed as containing an address that differs from object address.
  - Reclaim object memory.

Hazard Pointers (HP): Readers

- Reader:
  - Dereference pointer, loading address to object,
  - Store address to HP slot,
  - Memory barrier,
  - Dereference pointer, loading address to object (again!)
  - Validate that address was unchanged,
  - [use object]
  - Clear HP slot (store-release).

Reference Counters (refcount)

- Count number of references to an object,
- When decremented to 0, a release callback is invoked to reclaim memory.

Existence Guarantees Tradeoffs (RCU)

- RCU readers are fast, scale well,
- Long read-side critical sections can postpone reclaim, leading to high memory footprint.
- Read-side critical sections naturally prevent reclaim of all linked-list nodes across entire reader traversal.

Existence Guarantees Tradeoffs (HP)

- Hazard Pointer readers are fast, scale well,
- Reclaim of memory can be done immediately after readers stop using the HP address,
- Good fit when readers dereference immortal pointers (existence is guaranteed)

Existence Guarantees Tradeoffs (HP)

- Immediate reclaim of successor elements can cause issues for data structure traversal where elements can be concurrently removed and reclaimed. Work-around strategies:
  - Readers must retry from the beginning of traversal (closest predecessor immortal pointer),
  - By convention, readers can prevent reclaim of items being traversed until traversal is over. E.g. HP in list head, second list of items to reclaim.
    - Downsides: Potentially unbounded memory use. Not a good fit if elements are chained within multiple data structures.

Existence Guarantees Tradeoffs (refcount)

- Cache-line bouncing when readers increment or decrement the refcount concurrently.
- Memory-efficient, immediate reclaim.
- Existence of the object containing the reference counter must be guaranteed by another mechanism before the reference counter can be incremented.

Combining Existence Guarantees

- It is possible to transition from one mechanism to another while preserving existence guarantees:
  - RCU -> refcount
  - RCU -> HP
  - HP -> refcount
  - ...

hpref: Hazard Pointers with Reference Counter

- Use HP as reader fast-path,
- Fallback to refcount when no slots are available,
- Benefit from speed of HP, limited memory use of refcount,
- Fixed set of HP slots: 8 slots per CPU (a single cache line),
- Readers can promote their HP to refcount if they intend to keep the reference for a long time.

hpref: Hazard Pointers with Reference Counter

- hpref_synchronize(node):
  - Wait for single HP value to be unused.
- hpref_synchronize(NULL):
  - Wait for all prior HP to pass through quiescent state (NULL).
  - 2-phase wait scheme to ensure forward progress.
  - Reader tag HP slot low bit with current phase (0 or 1).
  - Wait for each slot to observe either NULL or slot value transition.
  - Useful for batching worker thread synchronize.

hpref benchmarks

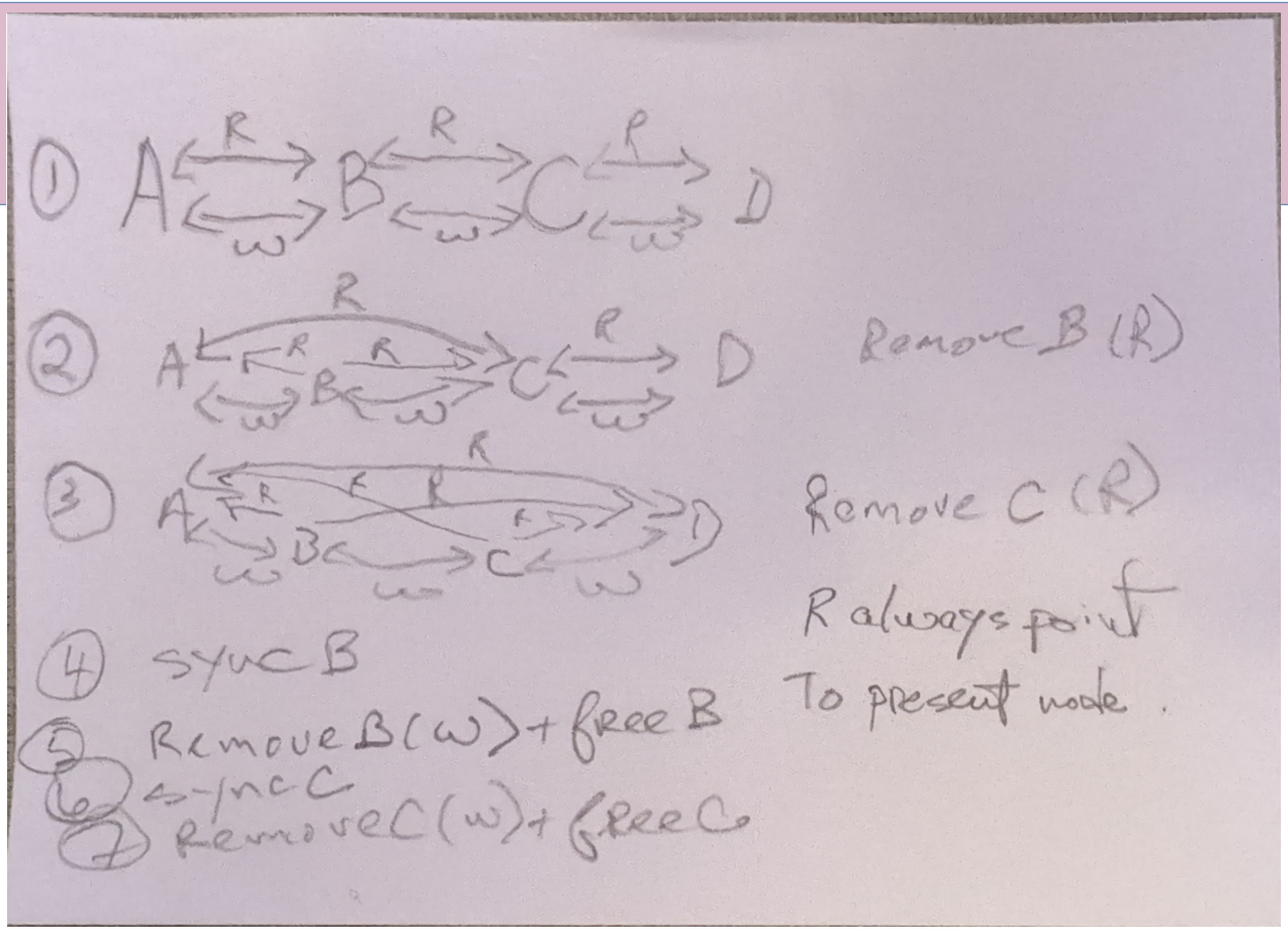
AMD Ryzen 7 PRO 6850U with Radeon Graphics

8 readers, 1 writer, 10s

```
test_rwlock                              nr_reads    190461165 nr_writes          12 nr_ops    190461177
test_mutex                               nr_reads    248594205 nr_writes    26088306 nr_ops    274682511
test_urcu_mb         (smp_mb)            nr_reads    829829784 nr_writes    18057836 nr_ops    847887620
test_perthreadlock                       nr_reads   1623365032 nr_writes     1244814 nr_ops   1624609846
test_hpref_benchmark (smp_mb)            nr_reads   1994298193 nr_writes    22293162 nr_ops   2016591355
test_hpref_benchmark (barrier/membarrier) nr_reads  15208690879 nr_writes     1893785 nr_ops  15210584664
test_urcu_bp         (barrier/membarrier) nr_reads  20242102863 nr_writes      599484 nr_ops  20242702347
test_urcu            (barrier/membarrier) nr_reads  20714490759 nr_writes      782045 nr_ops  20715272804
test_urcu_qsbr                           nr_reads  40774708959 nr_writes     3512904 nr_ops  40778221863
```

hpref (future work)

- Port from liburcu/librseq (userspace) to the Linux kernel,
- Apply to speed up heavy users of reference counts,
- Scheduler could promote HP to refcount on preemption,
- Rather than wait for HP to stop being used before reclaim, could instead make the reader fall-back to refcount (remotely).
  - This would provide forward progress guarantees for updater reclaim.
- Figure out if and how hpref can be applied traversal of data structures which are modified concurrently (e.g. HP linked-list).



HP linked-list

Questions / Comments ?

- References:
  - M. M. Michael, "Hazard pointers: safe memory reclamation for lock-free objects," in IEEE Transactions on Parallel and Distributed Systems, vol. 15, no. 6, pp. 491-504, June 2004
- Links:
  - https://lore.kernel.org/lkml/20240922102002.321008-1-mathieu.desnoyers@efficios.com/